Desarrollo del Software a partir de la Integración de sus Módulos

Análisis y Desarrollo de Software

Servicio Nacional de Aprendizaje (SENA)

Eliana Chacon

1 de noviembre de 2026

1. Introducción

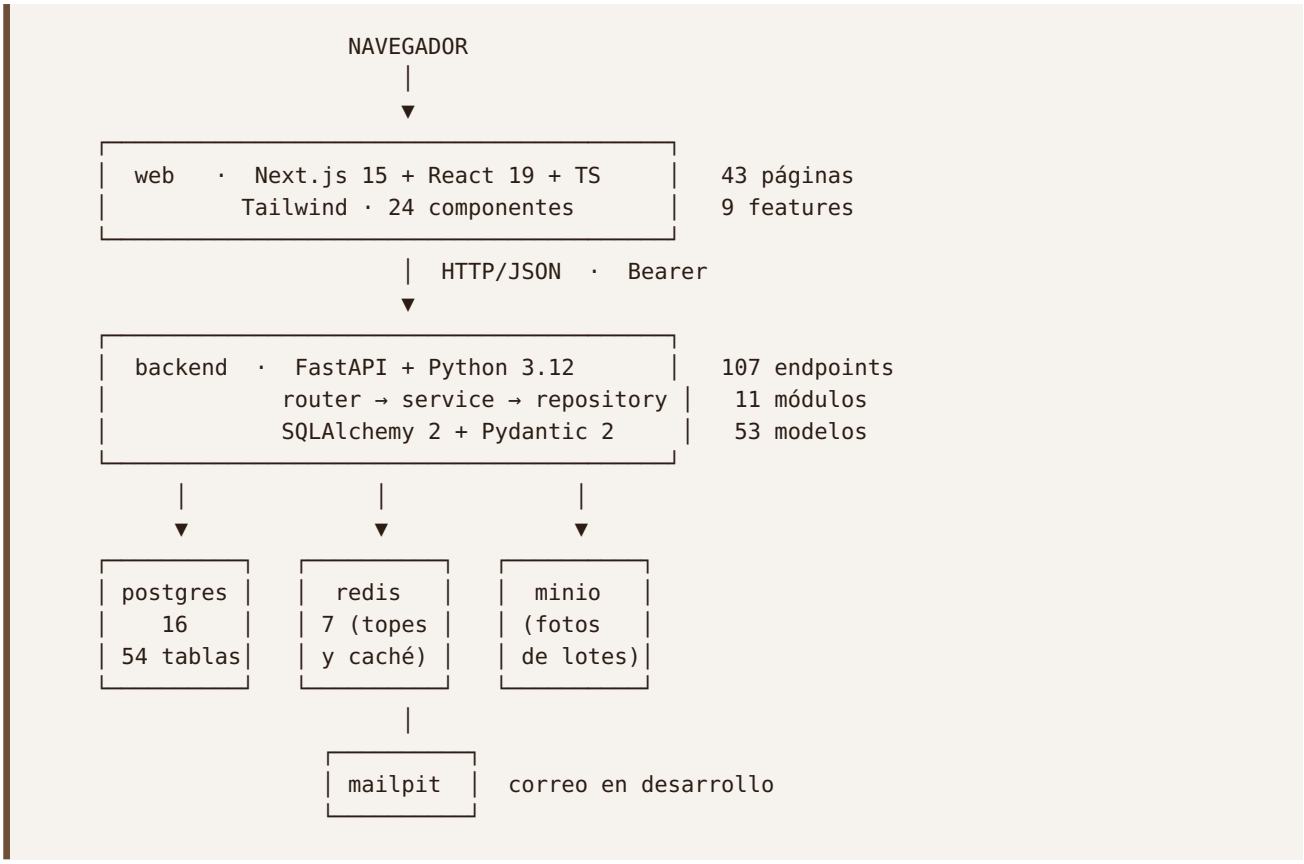
Las evidencias de GA7 construyeron piezas: un módulo de acceso a datos, una capa web con servlets, un módulo con framework, componentes de interfaz, servicios web. Esta evidencia documenta el paso siguiente y más difícil de demostrar: **que esas piezas forman un solo sistema que arranca, se comunica y se puede probar de punta a punta**.

La diferencia no es retórica. Un módulo puede estar bien escrito y no integrarse: porque su esquema no coincide con el de la base, porque dos capas se llaman en la dirección equivocada, o porque funciona en la máquina de quien lo escribió y en ninguna otra. Este documento no afirma que MONK GRAIN esté integrado: lo **comprueba**, y las comprobaciones están en el texto con su salida.

2. Objetivo

Codificar e integrar los módulos de MONK GRAIN en una aplicación orientada a la web, a partir de los requerimientos del sistema, identificando capas, mecanismos de seguridad, librerías y frameworks por capa; dividiendo en componentes reutilizables y paquetes con nombres claros; aplicando patrones de diseño; con pruebas unitarias por módulo, control de versiones, configuración de servidores y bases de datos, y documentación de los ambientes de desarrollo y pruebas.

3. El sistema, de un vistazo



Seis servicios en `docker-compose.yml`. Un solo comando los levanta, y esa es la primera prueba de integración que vale: si hiciera falta instalar PostgreSQL, Redis y MinIO a mano, el sistema estaría integrado solo en la máquina de quien lo escribió.

4. Las capas, y qué framework vive en cada una

Capa	Dónde	Librerías y frameworks	Responsabilidad
Presentación	`web/src/app/`, `web/src/components/`	Next.js 15, React 19, TypeScript, Tailwind, lucide-react	Dibujar y recoger. No decide reglas
Cliente de API	`web/src/features/*/api.ts`, `web/src/lib/api.ts`	`fetch` nativo	Hablar HTTP y traducir errores a `ApiError`
Entrada HTTP	`backend/app/modules/*/router.py`	FastAPI, Pydantic 2	Validar la forma, autorizar, traducir a llamadas del servicio
Negocio	`backend/app/modules/*/service.py`	Python puro	Las reglas. Es la única capa que decide
Datos	`backend/app/modules/*/repository.py`, `app/models/`	SQLAlchemy 2	Leer y escribir. No conoce HTTP
Infraestructura	`backend/app/core/`	Argon2, Redis, boto3, structlog	Lo transversal: seguridad, topes, archivos, bitácora

4.1 La regla de dependencia

`router → service → repository → models`, **en una sola dirección**. Un repositorio que importara el router cerraría el ciclo y haría imposible probar el servicio sin levantar un servidor. Es la regla que más se gana por respetarla: las 435 funciones de prueba del backend existen porque la lógica se puede llamar directamente.

La misma regla tiene espejo en el frontend: `features/trading/` corresponde a `modules/trading/`. Quien busca dónde se dibuja algo del mercado de dos sentidos sabe dónde mirar sin preguntar.

5. Los módulos

Módulo	End-points	Capas presentes	Qué resuelve
`auth`	10	router · service · repository	Registro, ingreso, renovación, recuperación
`catalog`	5	router · service · repository	Lo que se puede comprar
`cart`	9	router · service	La selección antes de pagar
`orders`	6	router · service	Pedidos y sub-órdenes
`returns`	10	router · service	Devoluciones y sus plazos
`reviews`	8	router · service	Reseñas de quien compró
`trading`	10	router · service	Oferta y demanda
`producers`	27	router · service	Panel del vendedor
`payments`	7	router · service	Liquidación al vendedor
`admin`	7	router · service	Aprobación, moderación, parámetros
`users`	7	**router**	Perfil, direcciones, consentimientos

> **Dos irregularidades que se encontraron midiendo, y que conviene nombrar.**
>
> La primera: `users` tiene router y **no tiene servicio**. Sus siete endpoints resuelven la lógica dentro del propio router. Funciona y está probado —36 pruebas entre perfil y direcciones— pero rompe la regla de §4.1, y el día que una regla de direcciones haya que usarla desde otro sitio habrá que extraerla.
>
> La segunda: solo `auth` y `catalog` tienen `repository.py`; los demás hacen el acceso a datos desde el servicio. No es un defecto en sí —SQLAlchemy ya es una capa de acceso— pero significa que la separación que describe §4 es menos estricta de lo que el diagrama sugiere. Se dice aquí en vez de dibujar una arquitectura más limpia que la real.

5.1 Tres carpetas de módulo vacías

```
backend/app/modules/content/     solo __init__.py
backend/app/modules/messaging/   solo __init__.py
backend/app/modules/shipments/   solo __init__.py
```

Tienen modelo y tabla — `conversations`, `messages`, `blog_posts`, envíos— y ningún endpoint detrás. Son alcance planeado y no construido. Están aquí porque un informe de integración que solo enumera lo que funciona no sirve para planear lo que falta.

6. Mecanismos de seguridad, por capa

Capa	Mecanismo	Qué impide
Navegador	`Content-Security-Policy`, `X-Frame-Options: DENY`	Que la página se embeba en un marco ajeno
	`X-Content-Type-Options: nosniff`, `Referrer-Policy`	Que el navegador adivine el tipo, y la fuga de rutas internas
Entrada HTTP	Validación con Pydantic 2	Que un cuerpo mal formado llegue a la lógica
	`require_permissions(...)` — 24 permisos	Que alguien haga lo que su rol no concede
	Topes de frecuencia en Redis, por cliente y por cuenta	Probar contraseñas y recorrer el padrón de correos
Negocio	404 en vez de 403 sobre recursos ajenos	Confirmar que un pedido existe
Datos	Argon2id para contraseñas	Que una tabla robada entregue las contraseñas
	Cifrado Fernet para los datos bancarios	Que una copia de la base entregue las cuentas
Despliegue	`requirements.lock` con 78 paquetes fijados	Que una dependencia cambie sola entre dos despliegues

Comprobado contra el servicio en marcha:

```
$ curl -D - -o /dev/null http://localhost:8000/health

x-content-type-options: nosniff
x-frame-options: DENY
referrer-policy: strict-origin-when-cross-origin
content-security-policy: default-src 'none'; frame-ancestors 'none';
                         base-uri 'none'; form-action 'none'
```

7. Patrones y componentes reutilizables

Patrón	Dónde	Qué problema resolvió
Repositorio	`auth/repository.py`, `catalog/repository.py`	Que el servicio no arme consultas a mano
Servicio	Los 10 `service.py`	Probar las reglas sin levantar un servidor
Inyección de dependencias	`Depends(get_session)`, `Depends(require_permissions(...))`	Que la prueba sustituya la sesión por una de pruebas
DTO	`app/schemas/`	Que un `{"role":"ADMIN"}` en el cuerpo no conceda nada
Manejador central de errores	`app/core/exceptions.py`	Que los 107 endpoints respondan el error con la misma forma
POST-Redirect-GET	`catalogo-web` (GA7-AA2-EV02)	Que recargar no repita el envío
Hook propio	`useProtectedResource`	25 líneas repetidas en 28 pantallas

En el frontend hay **24 componentes** repartidos en tres niveles: primitivas (`components/ui/`), transversales (`components/shared/`) y de dominio (`features/*/components/`). El criterio para extraer uno está documentado en GA7-AA4-EV02: se extrae cuando se repite, cuando encierra una regla del negocio o cuando tiene estado propio; **nunca porque sea grande**.

8. Base de datos y migraciones

PostgreSQL 16, **54 tablas**, con el esquema versionado en siete migraciones de Alembic que cuentan la historia del proyecto:

```
<base>      →  db2be3833cad   initial schema
            →  59c2f7a91062   order number sequence
            →  ae0cdd0bb859   seller notification data and withdrawal reason
            →  6cd4b72f44fd   returns flow: items, evidence, deadlines
            →  2840b05b463c   review reports moderation queue
            →  99716e5f6986   Pivote del dominio: de café tostado al detal
                              a pergamino seco por kilogramo
            →  4ad7359adcf8   Oferta y demanda: el comprador publica lo que
                              busca y el vendedor responde          (head)
```

8.1 La comprobación que de verdad importa

Que existan migraciones no prueba que el esquema y los modelos coincidan. Lo que lo prueba es pedirle a Alembic que compare:

```
$ docker compose exec backend alembic current
4ad7359adcf8 (head)

$ docker compose exec backend alembic check
No new upgrade operations detected.
```

Y el contraste directo entre los modelos y la base:

```
tablas en los modelos: 53
tablas en la base    : 54        (la de más es `alembic_version`)

EN LOS MODELOS Y NO EN LA BASE: ninguna
EN LA BASE Y NO EN LOS MODELOS: ninguna
```

Esto es lo que separa «hay migraciones» de «el esquema está integrado». Un modelo con una columna que la base no tiene no falla al arrancar: falla la primera vez que alguien consulta esa tabla, y para entonces ya está en producción. El CI ejecuta esta misma comprobación en cada subida, en el paso «Comprobar que no faltan migraciones».

9. Pruebas por módulo

453 pruebas en 23 archivos, con **87 % de cobertura** sobre 6.055 líneas.

Archivo	Pruebas	Módulo que cubre
`test_trading.py`	40	`trading`
`test_returns.py`	32	`returns`
`test_settlements.py`	27	`payments`
`test_addresses.py`	25	`users`
`test_producer_catalog.py`	25	`producers`
`test_reviews.py`	25	`reviews`
`test_admin.py`	21	`admin`
`test_catalog.py`, `test_pergamino.py`	40	`catalog`
`test_certifications.py`, `test_fulfillment.py`	38	`producers`
`test_checkout.py`, `test_receipt.py`	29	`cart`, `orders`
`test_auth_flow.py`, `test_rate_limit.py`	36	`auth`
`test_profile.py`, `test_wishlist.py`	22	`users`, `cart`
5 archivos de `tests/unit/`	72	`core`: seguridad, DANE, archivos, días hábiles
`test_performance.py`	3	Transversal

```
$ docker compose exec backend pytest -q --cov=app

TOTAL      6055   771   87%
453 passed in 206.09s
```

> Las funciones `def test_...` son **435** y las pruebas ejecutadas son **453**. La diferencia no es un error de cuenta: algunas están parametrizadas y una sola función produce varias pruebas. Se anotan las dos cifras porque cada una contesta una pregunta distinta —cuánto código de prueba hay, y cuántos casos se verifican—.

En el frontend, **53 pruebas** en 5 archivos. Dos de ellas no prueban componentes sino el proyecto: `navigation.test.ts` contrasta los enlaces del menú y el pie contra el árbol de páginas reales, y `quality.test.ts` vigila convenciones. Son pruebas de integración en el sentido más literal: fallan cuando dos partes dejan de corresponderse.

10. Integración continua

Cuatro trabajos en GitHub Actions, en cada subida y en cada propuesta de cambio:

Trabajo	Qué hace
`backend`	Levanta PostgreSQL, Redis y almacenamiento de objetos; linting, formato, tipos; **aplica las migraciones**; comprueba que no falten; 453 pruebas con cobertura
`frontend`	Tipos, linting, formato, 53 pruebas y compilación de producción
`docker`	Construye las dos imágenes de producción
`security`	Dependencias con vulnerabilidades conocidas, reglas de seguridad del código, y búsqueda de secretos en el historial

El flujo estaba escrito desde el principio y **nunca se había ejecutado**. Al publicar el repositorio falló tres veces seguidas, y las tres causas eran reales: faltaba declarar el almacenamiento de objetos, el lockfile del frontend se había generado en Alpine y los ejecutores son glibc, y las imágenes de MinIO dejaron de poder descargarse sin credenciales. Está contado en el README. Es la demostración más honesta de para qué sirve un CI: ninguno de los tres se veía desde la máquina de desarrollo.

11. Ambientes

	Desarrollo	Pruebas	Producción
Orquestación	`docker-compose.yml`	GitHub Actions + `conftest.py`	`docker-compose.prod.yml`
Base de datos	PostgreSQL 16 en contenedor	PostgreSQL de servicio, esquema por migraciones	PostgreSQL gestionado
Archivos	MinIO	`adobe/s3mock`	S3 o compatible
Correo	Mailpit (no sale nada)	—	Proveedor SMTP
Recarga	Sí, código montado	—	No, imagen fija
Secretos	`.env` local	Variables del flujo	Variables del entorno
Datos	Semilla + demostración	Creados por cada prueba	Reales

11.1 Levantar el sistema entero

```
$ git clone https://github.com/monjejuan448-ux/MONK-GRAIN-APP
$ cd MONK-GRAIN-APP
$ cp .env.example .env
$ docker compose up -d

$ docker compose exec backend alembic upgrade head
$ docker compose exec backend python -m app.db.seed
$ docker compose exec backend python -m app.db.demo_data

  http://localhost:3000     el sitio
  http://localhost:8000/docs  la API, con Swagger
  http://localhost:8025     el correo de desarrollo
```

> **Una trampa del entorno que cuesta horas.** `pytest` debe ejecutarse **de a uno**. Dos sesiones simultáneas comparten la base de pruebas y producen cientos de errores que no existen —326 la primera vez que pasó—. No es un defecto del código: es que el aislamiento de las pruebas es por transacción y no por base.

12. Control de versiones

Repositorio	github.com/monjejuan448-ux/MONK-GRAIN-APP
Rama por defecto	`main`
Flujo	Rama por tema → propuesta de cambio → revisión → fusión
Mensajes	Asunto en imperativo; el cuerpo explica por qué, no qué
Fuera del repositorio	`.env`, `target/`, `node_modules/`, `__pycache__`, los ZIP de entrega

13. Verificación de la integración

Lo anterior describe. Esto comprueba. Cada línea es una orden que se puede repetir.

Qué se comprueba	Cómo	Resultado
Los seis servicios arrancan	`docker compose ps`	6 running
El backend responde	`curl /health`	`{"status":"ok"}`
Los 107 endpoints están montados	`curl /openapi.json`	107
El esquema coincide con los modelos	`alembic check`	No new upgrade operations
Las reglas de negocio funcionan	`pytest`	453 pruebas, 87 %
El frontend compila y pasa	`tsc` , `lint` , `format` , `test`	53 pruebas, 4 puertas limpias
El frontend habla con el backend	Colección de Postman (GA7-AA5-EV04)	28 peticiones, 48 comprobaciones
Los enlaces no mienten	`navigation.test.ts`	Ningún enlace a página inexistente

14. Lo que no está integrado

Qué	Estado
`content` , `messaging` , `shipments`	Modelo y tabla, sin endpoints. Alcance planeado, no construido
Los cupones	El carrito los consume y ningún endpoint los crea
Proceso en segundo plano	No existe. Nada vigila el vencimiento de certificaciones ni los plazos de devolución
Módulo `users`	Sin capa de servicio: la lógica vive en el router
Despliegue público	`docker-compose.prod.yml` está escrito y **nunca se ha desplegado**
Los módulos Java de GA7	Corren aparte (8080, 8081, 8082) contra su propia base `monkgrain_java` . No están integrados con el producto, y es deliberado: son ejercicios de aprendizaje y no deben tocar los datos reales

15. Conclusión

Los once módulos del backend, los nueve del frontend y los cuatro servicios de infraestructura arrancan con un comando, se comunican por HTTP con tokens, comparten un esquema que Alembic verifica contra los modelos, y se prueban con 453 pruebas de backend y 53 de frontend que un CI ejecuta en cada subida.

De todo lo anterior, la comprobación que más dice sobre la integración no es el número de pruebas: es `alembic check` devolviendo «No new upgrade operations detected». Significa que lo que el código cree de la base y lo que la base es coinciden exactamente — y esa es la costura por donde los sistemas integrados se rompen en silencio.

Lo que falta está dicho en §14 sin suavizarlo, incluidas las dos irregularidades de arquitectura que aparecieron al medir para escribir este documento y que no se conocían antes: un módulo sin capa de servicio y tres carpetas de módulo vacías.